scaleLLMflow

General LLM Workflows for Scientific Quality Scales

Proyecto MQS

Version 0.3.5
2026-08-14

Package scaleLLMflow

Title General LLM Workflows for Scientific Quality Scales

Version 0.3.5

Depends R (>= 3.5.0)

Imports httr2, jsonlite, openssl, pdftools, stringr

License MIT + file LICENSE

URL https://github.com/ppmena/scaleLLMflow

Contents

1 Package scaleLLMflow

1.1 Description

Reusable helpers for registering scientific quality scales, resolving prompts by provider and model, sending article text to Gemini, OpenAI, or Claude, validating strict JSON responses, parsing item scores, comparing them with reviewed references, and writing audit and evidence files.

The package deliberately separates three layers: the formal scale definition in metadata, the operational prompt sent to the model, and the numeric coding used for analysis. The bundled MQS and PEDro registries provide provider-generic prompts for Gemini, OpenAI, and Claude, plus selected model-specific variants.

1.2 Two workflow modes

Free mode The model response is scored and preserved for inspection without assuming that a previous answer is correct.

Reference mode A reviewed score vector is supplied and every returned item is compared with it. This mode reports agreement and mismatches while preserving the model's evidence and reasoning.

1.3 Credentials and provider configuration

Gemini credentials are read from `GEMINI_API_KEY` or `GOOGLE_GEMINI_KEY`. OpenAI credentials are read from `OPENAI_API_KEY`; `OPENAI_PROJECT_ID` may also be supplied. A project-level `.Renviron` file is convenient for interactive work in RStudio and should not be committed. Claude credentials are read from `ANTHROPIC_API_KEY` or `CLAUDE_API_KEY`.

2 Registry and scale helpers

2.1 `available_scales`

Description
Lists scales available in the prompt registry.

Arguments

`registry_dir` Optional registry root. The bundled registry is used when omitted.

Value
A character vector of scale identifiers.

2.2 `available_provider_models`

Description
Queries the provider's live model catalogue. This function is independent of the scale registry and does not use hard-coded model lists.

Arguments

`provider` `gemini`, `openai`, `claude`, or an alias. The aliases `chatgpt` and `anthropic` are accepted where applicable.

`api_key` Optional in-memory key. Otherwise the provider environment variables are used.

`timeout` Request timeout in seconds.

`supported_only` For Gemini, retain models supporting `generateContent`.

Value
A data frame with provider, model identifier, and display name. OpenAI uses `OPENAI_API_KEY`; Gemini uses `GEMINI_API_KEY` or `GOOGLE_GEMINI_KEY`; Claude uses `ANTHROPIC_API_KEY` or `CLAUDE_API_KEY`.

2.3 `resolve_prompt`

Description
Resolves the most appropriate registered prompt and its metadata. Matching proceeds from the exact model to family, numeric-version, provider-specific generic, generic, and nearest available fallbacks. The provider-specific fallback folders are named `claude-generic`, `gemini-generic`, and `openai-generic`.

Arguments

`scale` Scale identifier.

`model` Requested model identifier.

`provider` Optional provider used to select its generic prompt.

`registry_dir` Optional registry root.

Value
A resolved registry record containing prompt text, metadata, scale definition, response schema, and provenance fields.

2.4 `audit_model_registry`

Description
Checks every scale and model registry entry for required files, valid metadata, a complete item definition, a coherent response schema, and prompt hashes. Exact duplicate prompts are grouped and a unification plan is reported.

Arguments

`registry_dir` Optional registry root.

`output_dir` Optional directory for audit CSV reports.

`apply_unifications` If `TRUE`, moves redundant prompt folders to a timestamped backup after reporting the plan.

Value
An audit result with issue and success counts, duplicate groups, and proposed or applied unifications.

3 Prompt and text preparation

3.1 `build_scale_prompt`

Description
Combines an operational scale prompt with the article text.

Arguments

`prompt_text` Registered prompt text.

`article_text` Article text to evaluate.

Value
The complete provider prompt as a character string.

3.2 `extract_article_text`

Description
Reads local Markdown or text articles and optionally removes the references section. PDF input is supported through `extract_pdf_text`. For PDF articles, `conversion = "llm"` generally produces better reading order and Markdown structure, especially for multi-column articles.

Arguments

`file_path` Local article path.

`filetype` One of `auto`, `txt`, `md`, or `pdf`.

`strip_references` Whether to remove references before model submission.

Value
Article text.

3.3 `extract_pdf_text`

Description
Extracts text from a local PDF using `pdftools`. With `conversion = "llm"`, an LLM reconstructs reading order and Markdown structure; For two-column PDFs, the conversion prompt explicitly requests complete left-column then right-column reading on each page. The model is selected independently with `provider` and `model` (or `conversion_provider` and `conversion_model` in `run_article`). this generally gives better results for complex and multi-column articles. If the extracted text exceeds `max_chars` (50,000 by default), it is split at safe line boundaries, converted in several calls, and reassembled into one Markdown file.

Arguments

`pdf_path` Local PDF path.

`strip_references` Whether to remove references.

Value
Article text.

3.4 `convert_article_markdown_llm`

Description
Converts extracted article text into faithful Markdown with an LLM, reconstructing reading order, headings, and clearly identified tables. This generally improves results for complex PDF layouts.

Value
Markdown text returned by the configured provider.

3.5 `strip_references_section`

Description
Removes a detected references or bibliography section.

Value
Text with the trailing references section removed when detected.

4 Parsing and provider calls

4.1 `parse_scale_scores`

Description
Validates and parses a strict JSON scale response. Fenced JSON is normalized in a controlled way before validation. The registered metadata defines required items, allowed values, excluded items, and total-score rules.

Arguments

`text` Raw model response.

`items` Expected item identifiers.

`metadata` Optional resolved metadata and JSON Schema.

Value
A named numeric vector of item scores. When metadata is supplied, the response is validated against its declared schema before scores are returned.

4.2 `run_llm`

Description
Calls Gemini through the Interactions API, OpenAI through the Responses API, or Claude through Anthropic's Messages API. It applies rate limiting, retries transient failures with exponential backoff, records detailed errors, and returns the raw response needed for auditability.

Arguments

`prompt` Prompt sent to the provider.

`provider` `gemini`, `openai`, `chatgpt`, `claude`, or `anthropic`.

`model` Provider model identifier.

`temperature` Sampling temperature; zero is recommended for reproducibility.

`top_p` Gemini nucleus sampling parameter; ignored by OpenAI and Claude.

`timeout` Timeout per attempt, in seconds.

`api_key` Optional in-memory API key.

`project_id` Optional OpenAI project id.

`max_retries` Maximum transient-failure retries.

`retry_wait_seconds` Initial retry delay.

`retry_backoff` Retry-delay multiplier.

`rate_limit_seconds` Minimum delay between requests in the current R process.

Value
A provider response object containing text, request metadata, model and parameter provenance, hashes, input size, and error details when relevant.

5 Article and dataset workflows

5.1 `run_article`

Description
Runs one local article through a registered scale prompt. It writes raw audit output plus convenient score and evidence CSV files.

Important arguments

`article_path` Local article file.

`scale` Scale identifier.

`provider` Provider identifier.

model Model identifier.

output_dir Output directory.

reference_scores Named reviewed scores; activates reference validation.

validation_mode `free` or `reference`.

temperature, top_p, timeout Provider parameters.

max_retries, retry_wait_seconds, retry_backoff, rate_limit_seconds Reliability controls.

Value
A result containing parsed scores, totals, evidence, reasons, validation comparisons, provenance, and output paths.

5.2 run_dataset

Description
Processes all supported local articles in a directory and consolidates scores, evidence, audit information, and errors. A semicolon-separated reference CSV with `ID` and item columns activates dataset reference mode.

Value
A dataset result with per-article records and consolidated output paths.

6 Bundled scales and registry layout

The bundled registry currently contains the MQS and PEDro scales. Each provider generic folder contains exactly two files:

```
inst/scales/<scale>/<provider>-generic/
  prompt.md
  metadata.json
```

The metadata file is authoritative for item identifiers, allowed decisions, included items, total-score rules, and the response schema. A model-specific folder may be added only when its prompt has been separately tuned or validated.

PEDro reports all 11 items, but item 1 is excluded from the official total. MQS uses its declared 10-item scoring definition. Do not infer a total from the prompt text; use the metadata and package parser.

7 Examples

The following example is suitable for an interactive RStudio session:

```
library(scaleLLMflow)
results <- run_dataset(
  articles_dir = "examples/03_pedro_free/articles",
  scale = "pedro",
  provider = "gemini",
  model = "gemini-3.6-flash-lite",
```

```
  output_dir = "results/pedro-gemini",
  validation_mode = "free",
  temperature = 0,
  top_p = 0.1
)
```

For Claude, configure `ANTHROPIC_API_KEY` or `CLAUDE_API_KEY` and run the bundled example. The scale and model are explicitly defined in the script:

```
Rscript examples/05_claude_free/run.R
```

Edit `scale` and `model` in that script to switch between MQS/PEDro or another Claude model.

For comparison against reviewed scores:

```
results <- run_dataset(
  articles_dir = "articles",
  scale = "mqs",
  provider = "openai",
  model = "gpt-4.1-mini",
  output_dir = "results/reference",
  reference_csv = "mqs_reference.csv",
  validation_mode = "reference"
)
```

8 Index